

Title : Lexical Analysis Using C Programming

Objectives

- **Program 1:** Identify whether each character in an input word is a vowel or a consonant.
- **Program 2:** Detect and display single-line (//) and multi-line (/* ... */) comments from a block of C source code.
- **Program 3:** Read an input string and differentiate uppercase (capital) letters from lowercase (small) letters.
- **Program 4:** Break down an execution statement into tokens: Keywords, Identifiers, Operators, and Numbers.

Theory

Lexical analysis is the first phase of a compiler. It reads the source code character by character and groups them into meaningful token categories.

- **Character Categorization (Programs 1 & 3):** Text parsing relies directly on ASCII boundaries. Uppercase letters span 65–90 ('A'–'Z'), while lowercase letters span 97–122 ('a'–'z'). Vowels are separated from consonants by testing matching rules against the specific character set: a, e, i, o, u (and their uppercase equivalents).
- **Comment Stripping (Program 2):** Comments are non-executable text segments. Single-line comments begin with // and terminate at a newline (\n). Multi-line comments open with /* and continue dynamically across character streams until the closing */ sequence is found.
- **Tokenization (Program 4):** A string buffer collects alphanumeric characters until it reaches a delimiter (space, tab, or semicolon). At this boundary, the collected word is checked against a list of reserved symbols to find **Keywords** (int, if, etc.), checked for numerical values to find **Numbers**, or classified as an **Identifier**. Mathematical or logical symbols are immediately categorized as **Operators**.

Programs

Program 1: Identification of Vowels and Consonants

```
#include <stdio.h>

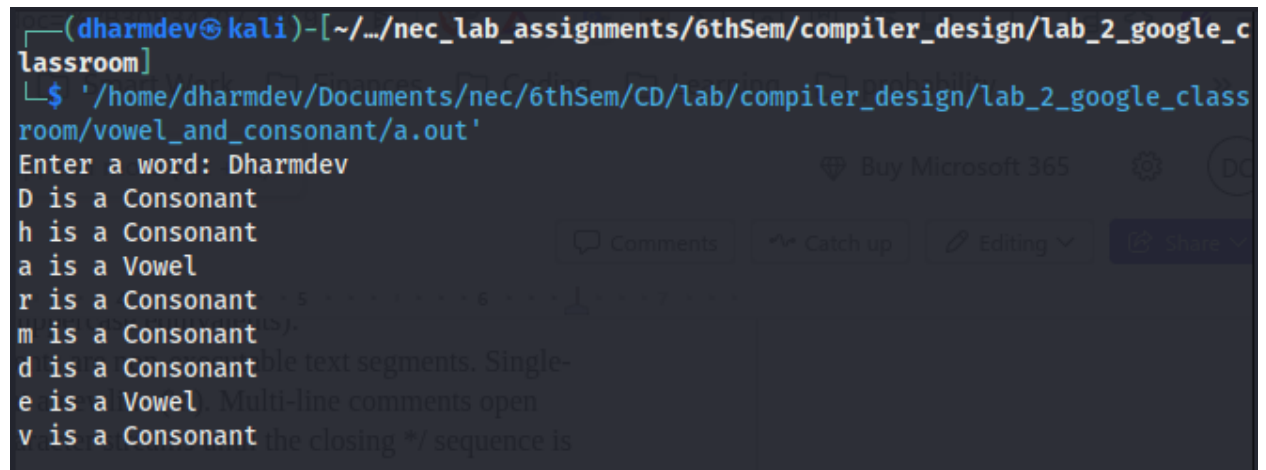
int main() {
    char str[100];
    printf("Enter a word: ");
    scanf("%s", str);
```

```

for (int i = 0; str[i] != '\0'; i++) {
    char ch = str[i];
    if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U') {
            printf("%c is a Vowel\n", ch);
        } else {
            printf("%c is a Consonant\n", ch);
        }
    } else {
        printf("%c is not a letter\n", ch);
    }
}
return 0;
}

```

Output:



```

(dharmdev@kali) - [~/.../nec_lab_assignments/6thSem/compiler_design/lab_2_google_classroom]
$ /home/dharmdev/Documents/nec/6thSem/CD/lab/compiler_design/lab_2_google_classroom/vowel_and_consonant/a.out
Enter a word: Dharmdev
D is a Consonant
h is a Consonant
a is a Vowel
r is a Consonant
m is a Consonant
d is a Consonant
e is a Vowel
v is a Consonant

```

Program 2: Identification of Single-Line and Multi-Line Comments

```

#include <stdio.h>

int main() {
    char code[500];
    printf("Enter C code (Type '#' at the end and press Enter to finish):\n");
    scanf("%[^#]", code);

    for (int i = 0; code[i] != '\0'; i++) {
        if (code[i] == '/' && code[i+1] == '/') {
            printf("Single-line Comment: ");
            while (code[i] != '\n' && code[i] != '\0') {

```

```

        printf("%c", code[i]);
        i++;
    }
    printf("\n");
}
else if (code[i] == '/' && code[i+1] == '*') {
    printf("Multi-line Comment: ");
    printf("%c%c", code[i], code[i+1]);
    i += 2;
    while (code[i] != '\0' && !(code[i] == '*' && code[i+1] == '/')) {
        printf("%c", code[i]);
        i++;
    }
    if (code[i] != '\0') {
        printf("*/\n");
        i++;
    }
}
}
return 0;
}

```

Output:

```

(dharmdev@kali) - [~/nec_lab_assignments/6thSem/compiler_design/lab_2_google_class
lassroom]
$ '/home/dharmdev/Documents/nec/6thSem/CD/lab/compiler_design/lab_2_google_class
room/single_line_and_multiline_comment/a.out'
Enter C code (Type '#' at the end and press Enter to finish):
//main method
int main(){
    printf("hello");
    /* above one
       is multiline comment
    */
    return 0;
}
#
Single-line Comment: //main method
Multi-line Comment: /* above one
                     is multiline comment
                     */

```

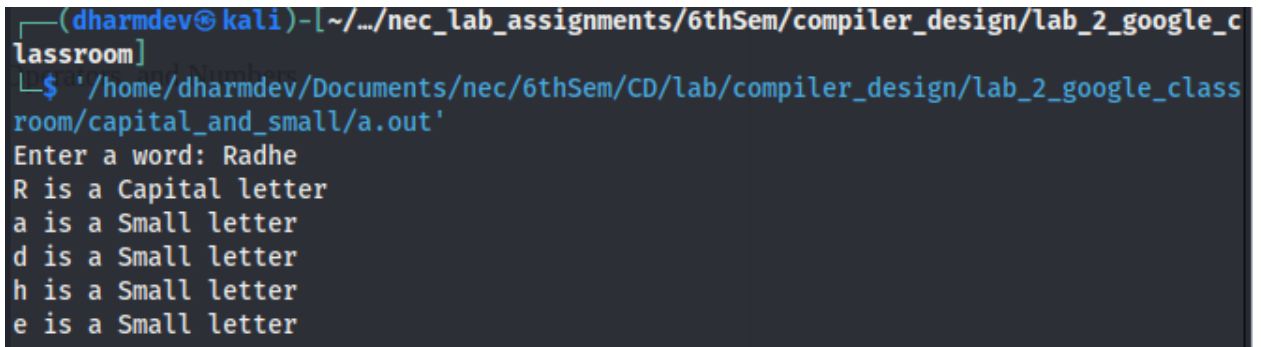
Program 3: Identification of Capital and Small Letters

```
#include <stdio.h>

int main() {
    char str[100];
    printf("Enter a word: ");
    scanf("%s", str);

    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] >= 'A' && str[i] <= 'Z') {
            printf("%c is a Capital letter\n", str[i]);
        } else if (str[i] >= 'a' && str[i] <= 'z') {
            printf("%c is a Small letter\n", str[i]);
        }
    }
    return 0;
}
```

Output:



```
(dharmdev@kali)-[~/.../nec_lab_assignments/6thSem/compiler_design/lab_2_google_c
lassroom]
└─$ ./../home/dharmdev/Documents/nec/6thSem/CD/lab/compiler_design/lab_2_google_class
room/capital_and_small/a.out
Enter a word: Radhe
R is a Capital letter
a is a Small letter
d is a Small letter
h is a Small letter
e is a Small letter
```

Program 4: Recognition of Keywords, Identifiers, Operators, and Numbers

```
#include <stdio.h>

int match(char str1[], char str2[]) {
    int i = 0;
    while (str1[i] != '\0' && str2[i] != '\0') {
        if (str1[i] != str2[i]) return 0;
        i++;
    }
    return (str1[i] == '\0' && str2[i] == '\0');
}
```

```

int isKeyword(char buffer[]) {
    if (match(buffer, "int") || match(buffer, "float") || match(buffer, "char") ||
        match(buffer, "if") || match(buffer, "else") || match(buffer, "while") ||
        match(buffer, "return")) {
        return 1;
    }
    return 0;
}

int main() {
    char statement[100], buffer[50];
    int j = 0;

    printf("Enter a statement (e.g., int x = 10 + y): ");
    scanf("%s", statement);

    for (int i = 0; statement[i] != '\0'; ++i) {
        char ch = statement[i];

        if (ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '=' || ch == '<' || ch == '>') {
            printf("Operator: %c\n", ch);
        }

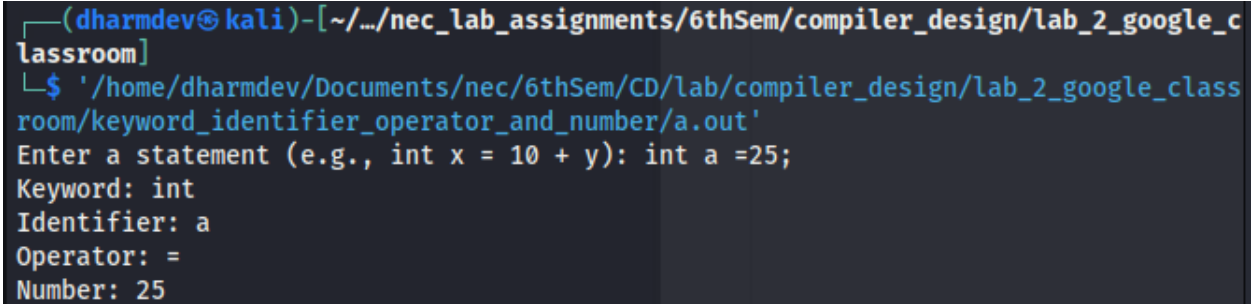
        if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z') || (ch >= '0' && ch <= '9') || ch
        == '_') {
            buffer[j++] = ch;
        }
        else if ((ch == ' ' || ch == '\t' || ch == ';') && j != 0) {
            buffer[j] = '\0';
            j = 0;

            if (isKeyword(buffer)) {
                printf("Keyword: %s\n", buffer);
            } else if (buffer[0] >= '0' && buffer[0] <= '9') {
                printf("Number: %s\n", buffer);
            } else {
                printf("Identifier: %s\n", buffer);
            }
        }
    }
}

```

```
    return 0;
}
```

Output:



```
(dharmdev@kali) - [~/.../nec_lab_assignments/6thSem/compiler_design/lab_2_google_classroom]
└─$ './$ "/home/dharmdev/Documents/nec/6thSem/CD/lab/compiler_design/lab_2_google_classroom/keyword_identifier_operator_and_number/a.out'
Enter a statement (e.g., int x = 10 + y): int a =25;
Keyword: int
Identifier: a
Operator: =
Number: 25
```

Conclusion

This lab demonstrates how raw character input streams can be tokenized into organized syntactic categories via direct conditional logic in C. Using basic loops and array indexing without high-level string libraries provides a clear view of how a compiler's front-end scanner performs lexical validation at a foundational level.